

计算机

组成原理与系统结构

主讲人：王蓝菁 | 学院：信息工程学院 | 时间：

第八章 互连及输入输出组织

When the night is darkest, dawn is nearest. Nothing is impossible for a willing heart.

本章主要内容

- 8.1 外部设备的分类与特点
- 8.2 常用输入输出设备
- 8.3 外设与CPU和主存的互连
- 8.4 I/O数据传送控制方式



输入输出设备与特性

- **输入输出设备**是计算机与人或者机器系统进行数据交互的装置，用于实现计算机内部二进制信息与外部不同形式信息的转换，简称**外部设备或外设**
 - **输入设备**：负责将数据、文字、图像、声音、电信号等转换成计算机可以识别的二进制信息，如键盘、鼠标、扫描仪、摄像头等；
 - **输出设备**：负责将计算机处理结果转换成数字、文字、图形、图像、声音或电信号，如显示器、打印机等；
 - **输入输出设备**：既能输入也能输出，如磁盘、网卡等。
- **输入输出设备特性**
 - **异步性、实时性、独立性**

输入/输出系统的组成与功能

- **外部设备、接口部件、总线以及相应的管理软件统称为计算机的输入/输出系统，简称I/O系统**

- **I/O硬件**

- ◆ 外设、控制器、I/O接口、总线

- **I/O软件**

- ◆ OS无关库，设备无关库，驱动

- **主要功能**

- 完成计算机内部二进制信息与外部多种信息形式间的交流
- 保证CPU能够正确选择I/O设备并实现对其控制，与数据传输
- 利用数据缓冲、合适的数据传送方式，实现主机外设间速度匹配



本章主要内容

- 8.1 外部设备的分类与特点
- 8.2 常用输入输出设备
- 8.3 外设与CPU和主存的互连
- 8.4 I/O数据传送控制方式



I/O接口定义与功能

■ I/O接口：连接总线与I/O设备的物理和逻辑界面

- 既包括**物理连接电路**，也包括**软件逻辑接口**
- 所有设备均通过 I/O接口（总线接口）与总线相连
- CPU使用设备地址经总线与I/O接口通信访问I/O设备
- **标准接口**有利于提升I/O系统的独立性，降低连接复杂度

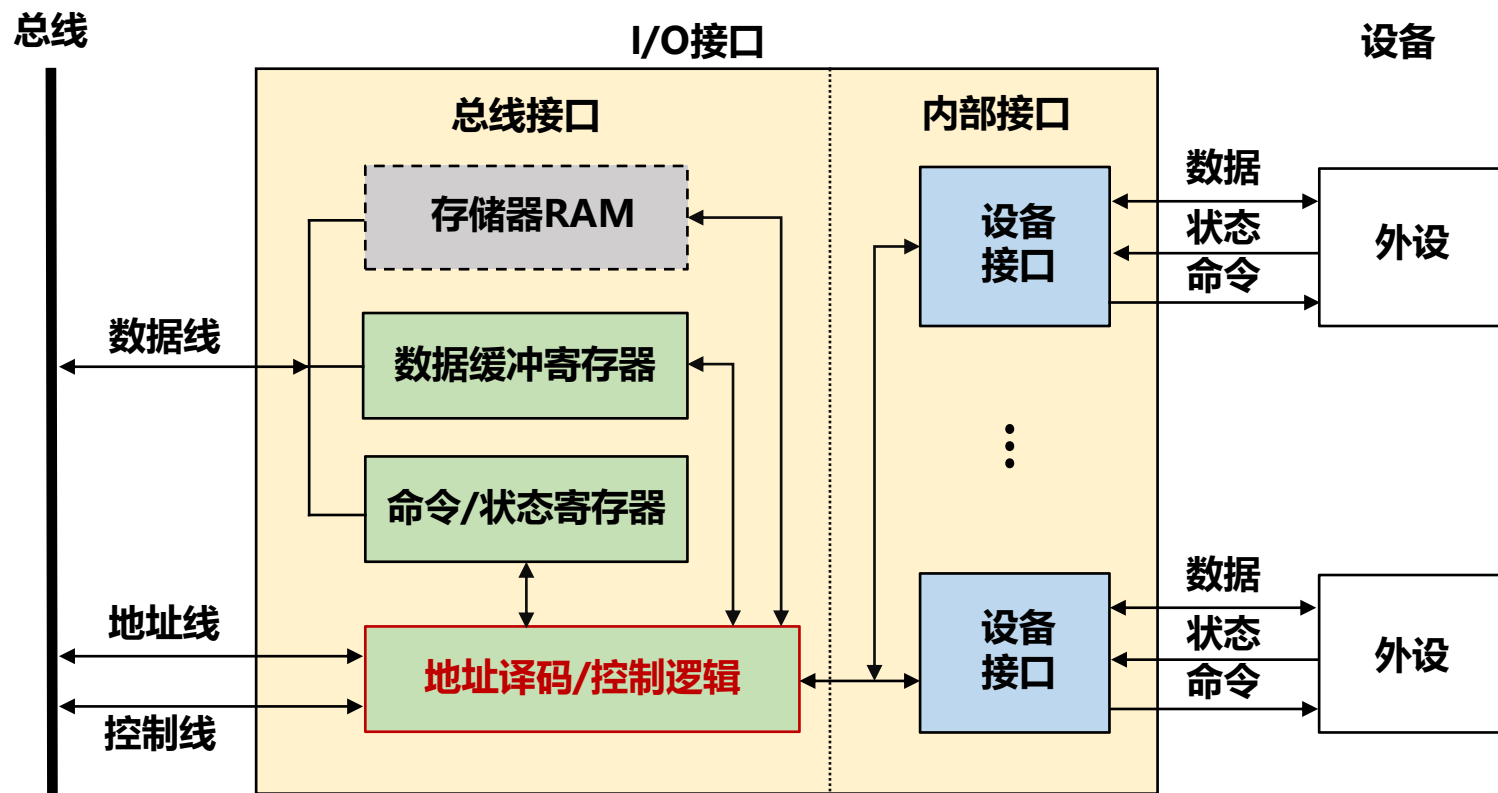
■ I/O接口功能

- **设备寻址、数据交互、设备控制**
- **状态检测、数据缓冲、格式转换**

I/O接口定义与功能

- **设备寻址：**接收CPU送来的地址码，选择接口中的寄存器供CPU访问。
- **数据交互：**通过I/O接口与主机之间、I/O接口与设备之间的通信来完成。
- **设备检测：**
- **状态检测：**提供状态寄存器，以保存各种错误或状态信息供CPU查用。
- **数据缓冲：**提供数据缓冲寄存器，以达到主机和外设工作速度的匹配。
- **格式转换：**提供数据格式转换部件使通过外部接口得到的数据转换为内部接口需要的格式，或在相反的方向进行数据格式转换。

I/O接口结构

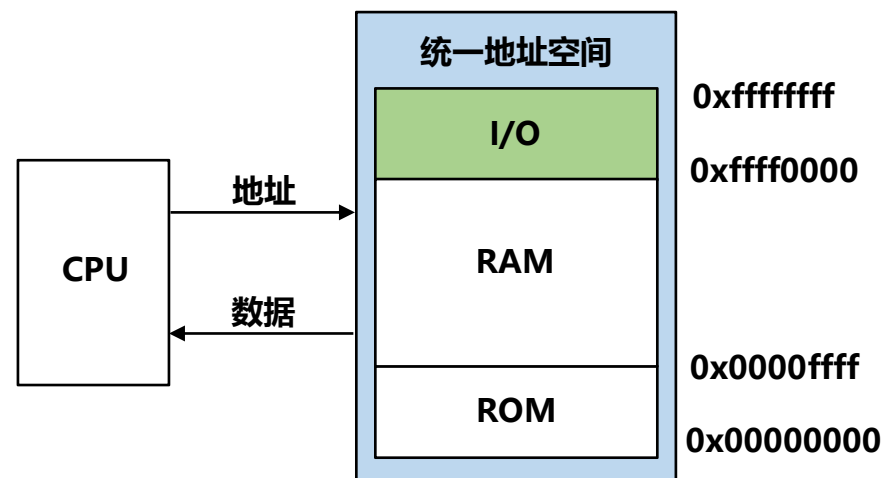


I/O接口的功能

- 设备寻址
- 数据交互
- 设备控制
- 状态检测
- 数据缓冲
- 格式转换

I/O接口编址

- 对I/O端口读写，就是向I/O设备送出命令或从设备取得状态或读/写设备数据
- 一个I/O控制器可能会占有多个端口地址
- I/O端口必须编号后，CPU才能访问它
- I/O设备的寻址方式就是I/O端口的编号方式
- 统一编址
 - 内存映射编址 (Memory-mapped)
 - 外设地址与内存地址统一编址，同一个地址空间
 - 低端地址→主存单元，高端地址→I/O端口
 - 不需要设置专用的I/O指令
 - 采用访存指令访问外设，具体访问什么设备取决于地址



I/O接口编址

■ 独立编址

- 端口映射编址 (Port-mapped)
- I/O地址空间与主存地址空间相互独立
- I/O地址又称为I/O端口
- 不同设备中的不同寄存器和存储器都有唯一的端口地址
- 使用I/O指令访问外设
- 需设置标志区分访问的对象

$$M/\overline{IO} \left\{ \begin{array}{l} =1 \text{ 访问存储器} \\ =0 \text{ 访问I/O端口} \end{array} \right.$$

I/O接口分类

■ 按数据传送方式

- 并行、串行接口

■ 按接口的灵活性

- 可编程接口、不可编程接口

■ 按通用性：通用、专用接口

■ 按总线传输的通信方式：同步、异步接口

■ 按访问外设的方式

- 直接传送方式、程序控制方式、程序中断方式、DMA及通道处理机接口

本章主要内容

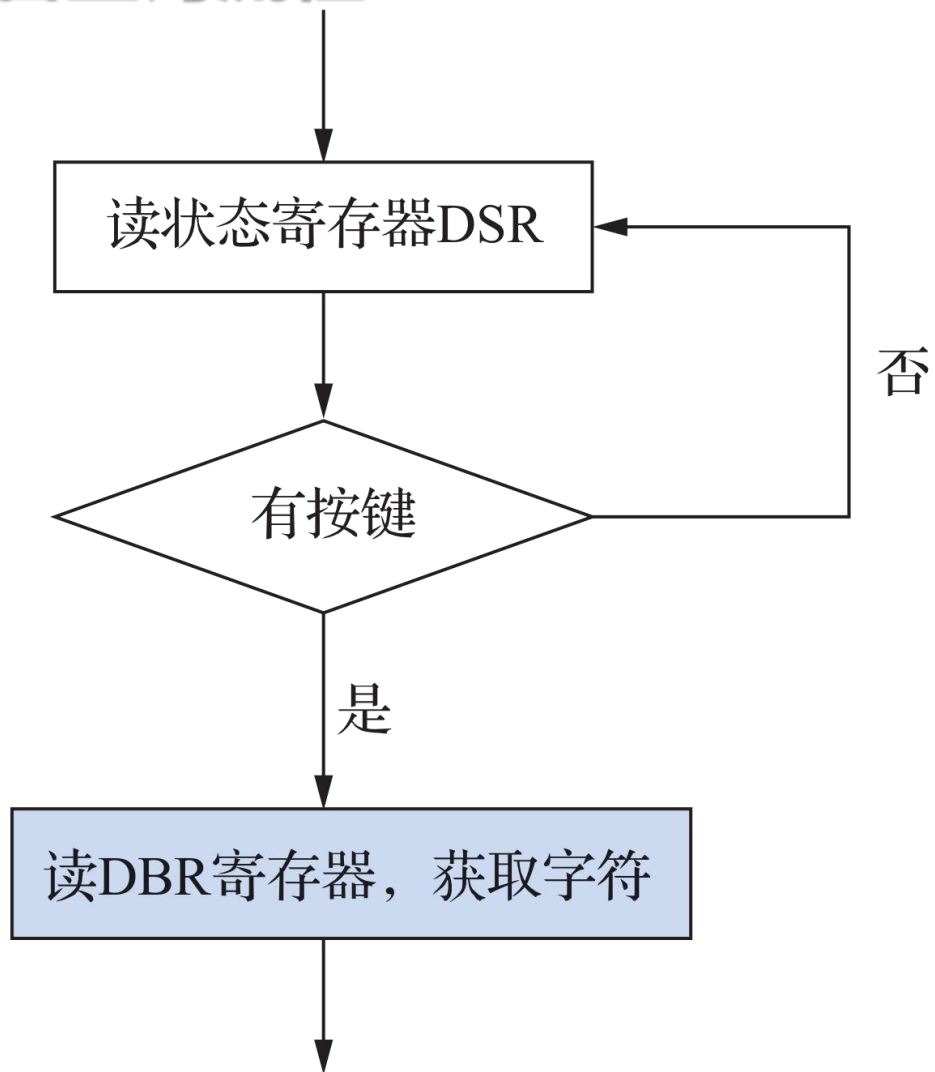
- 8.1 外部设备的分类与特点
- 8.2 常用输入输出设备
- 8.3 外设与CPU和主存的互连
- 8.4 I/O数据传送控制方式



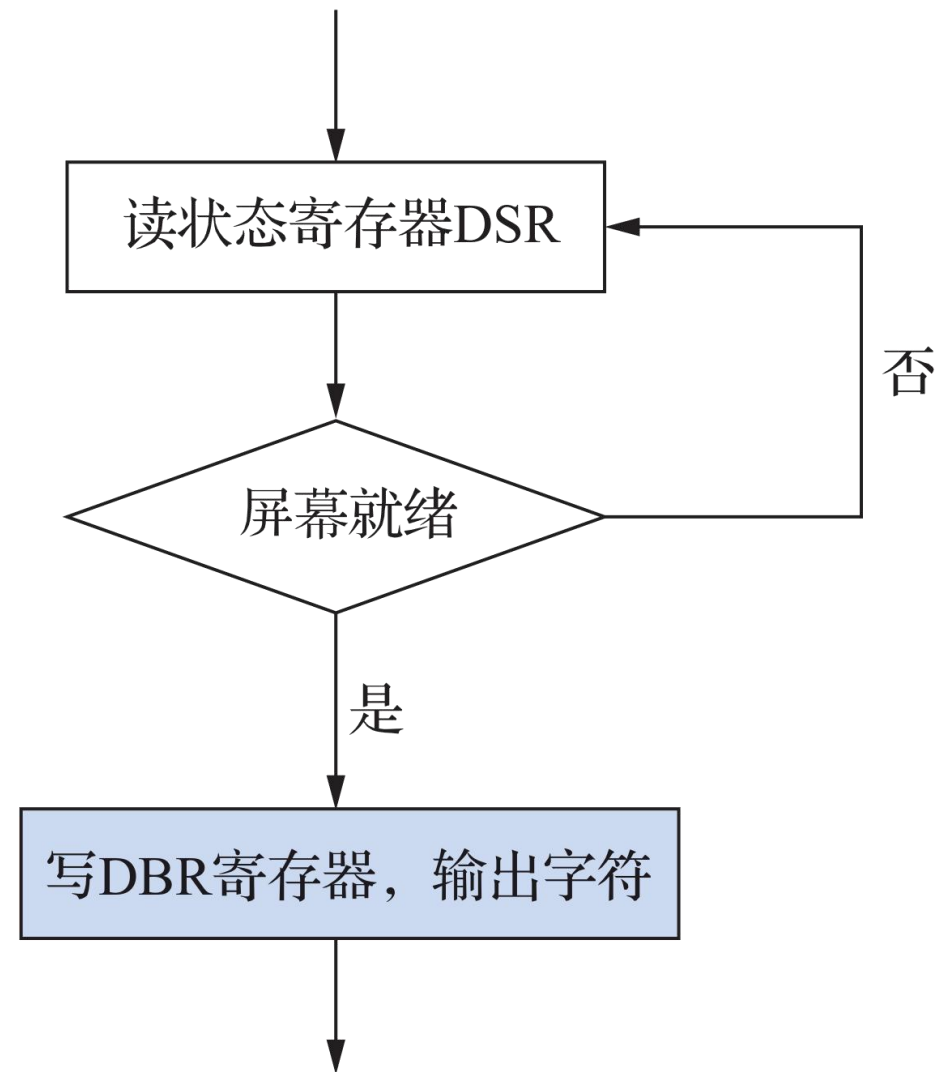
I/O传输方式

- 主机与外围设备之间的I/O操作通常有4种不同的工作模式：
 - 程序直接控制I/O方式(PIO)
 - 中断控制I/O方式
 - DMA方式
 - IOP或PPU方式(专用处理器/处理机)
- 各种方式工作模式的核心差异在于：在数据I/O过程中，主CPU承担的任务不同

简单设备查询流程

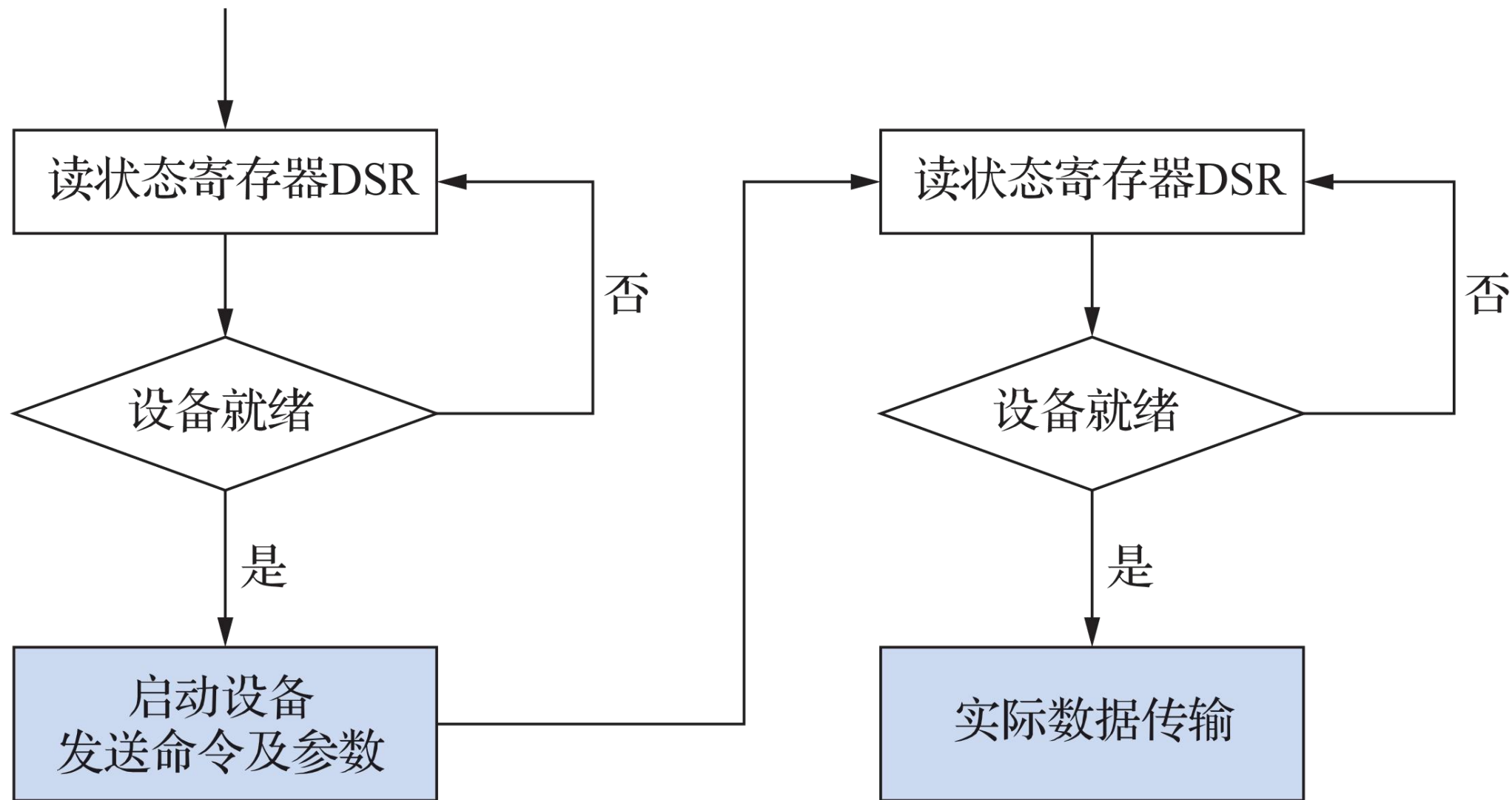


(a) 键盘程序查询流程



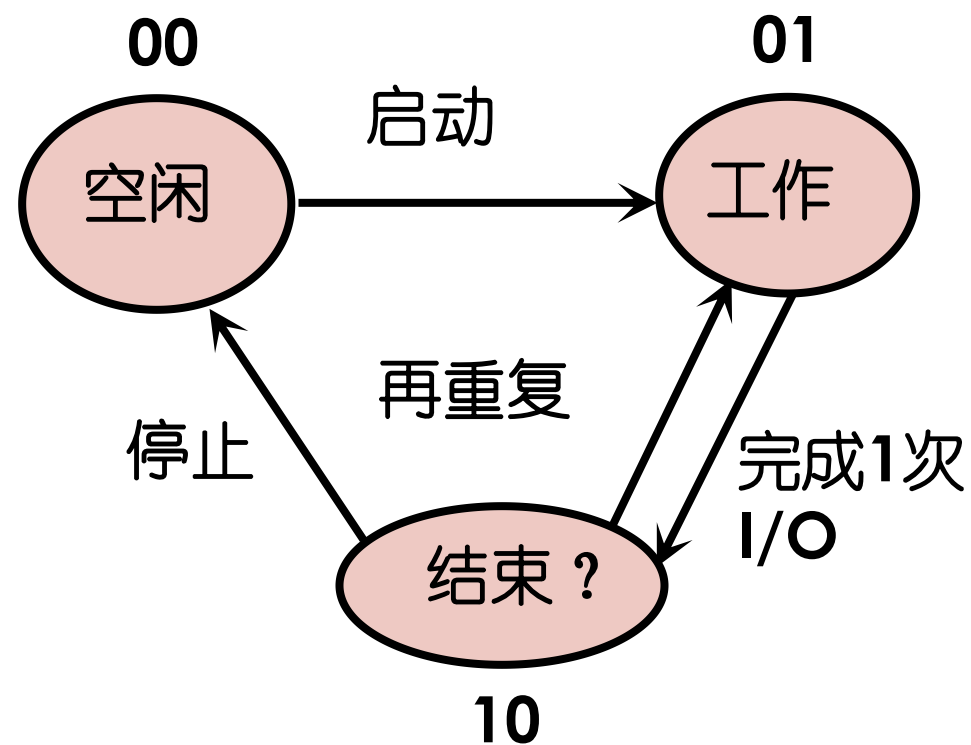
(b) 字符终端程序查询流程

复杂设备查询流程



程序直接控制I/O方式

- 特征：主CPU执行I/O程序实现主-外的数据I/O

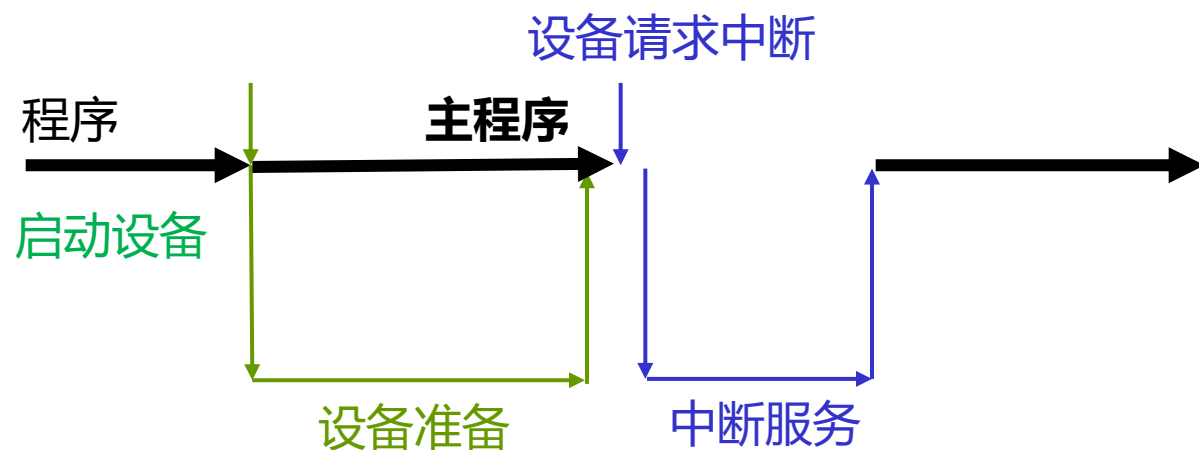


程序直接控制I/O方式特点

- 需要CPU执行专用的I/O程序
- CPU不断查询外设状态，进而控制具体的数据I/O过程（主机 $\longleftrightarrow$ 外设）
- 数据I/O过程中主CPU无法执行其它计算任务，对其它外设的请求响应慢
- 主CPU完全束缚于简单的数据I/O控制，利用率低
- 工作方式：完全串行工作方式或部分串行，CPU用100%的时间为I/O服务

中断基本概念

- CPU暂时中止现程序的执行，转去执行为某个**随机事件**服务的中断处理子程序，处理完后自动恢复原程序的执行
 - **原因：**不处理会体验不好，会丢失数据，甚至造成灾难
- 实现主机和外设准备阶段的并行工作
 - 避免重复查询外设状态、提升工作效率



中断优势

■ 提高了CPU的使用效率

- 主动告知机制避免了反复查询设备状态
- 仍需CPU占用（中断服务子程序运行时间+中断开销）

■ 适合随机出现的服务

■ 需要专门的硬件

中断实质与特点

(1) 实质

程序切换

{ 方法：保存断点，保护现场；
恢复现场，返回断点。
时间：一条指令结束时切换。

保证程序的完整性

(2) 特点

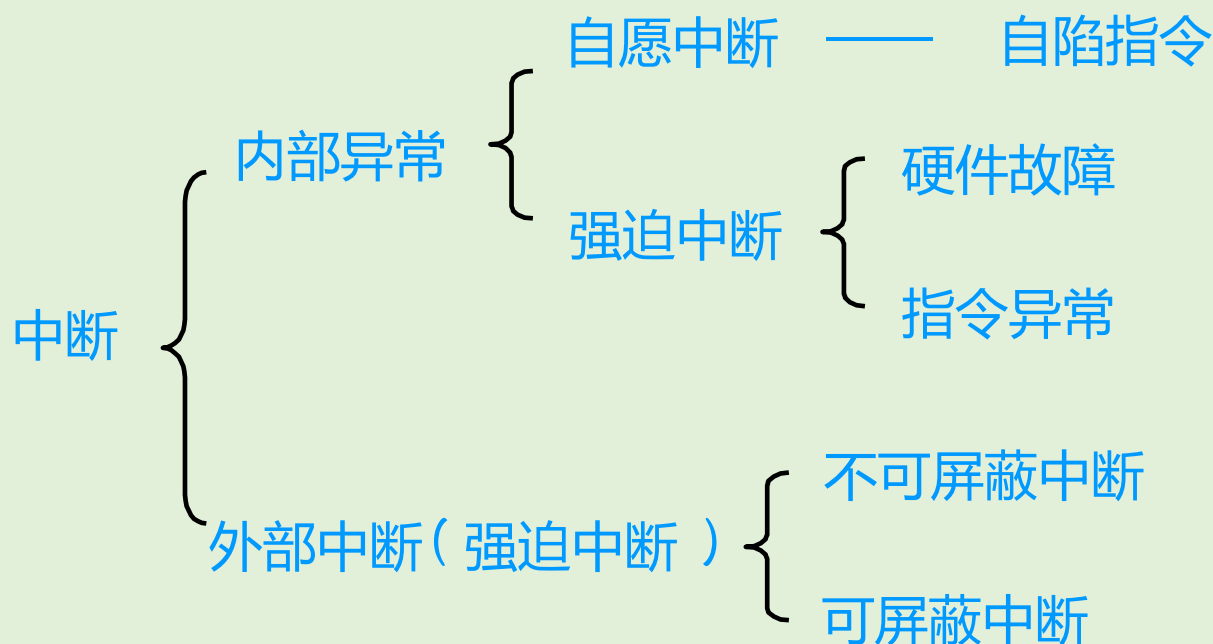
随机性

{ 随机发生的事态(按键、故障)
有意调用,随机请求与处理的事态(调用打印机)
随机插入的事态(软中断指令插入程序任何位置)

中断的分类与作用

- 中断技术赋予计算机应变能力，将有序的运行和无序的事件统一起来，大大增强了系统的处理能力

- 主机外设并行工作
- 程序调试
- 故障处理
- 实时处理
- 人机交互

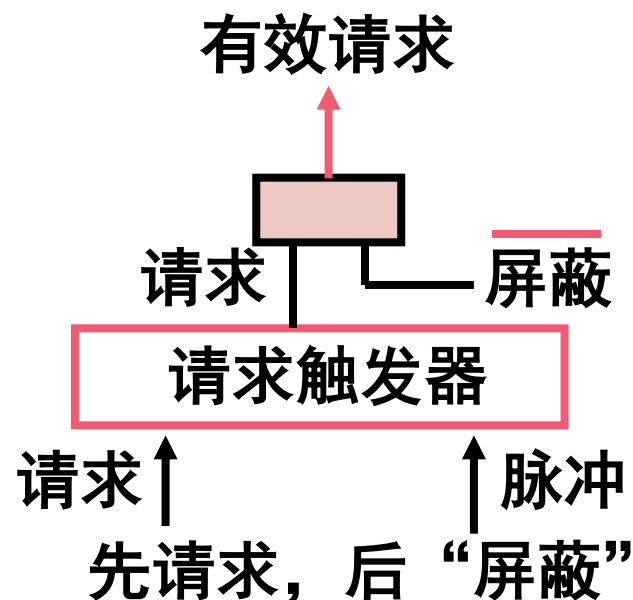
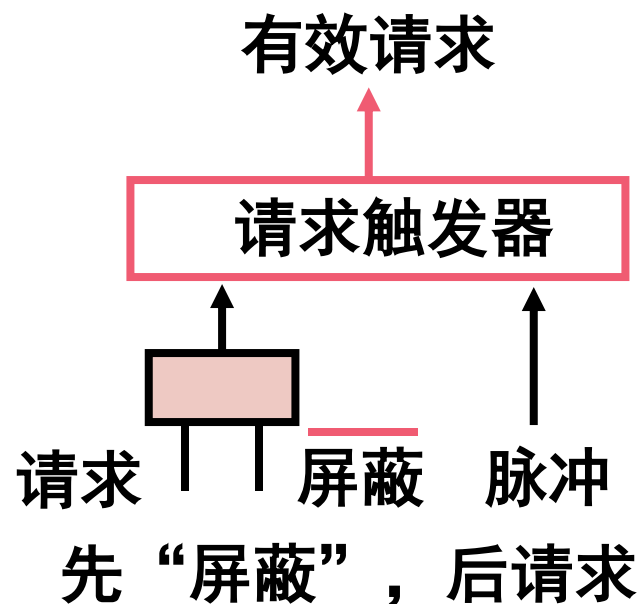


中断全过程（外中断）

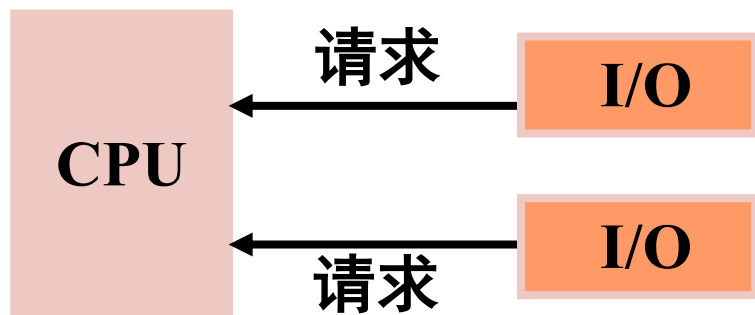
1. 中断请求的提出与传递

(1) 能产生中断请求的前提条件？

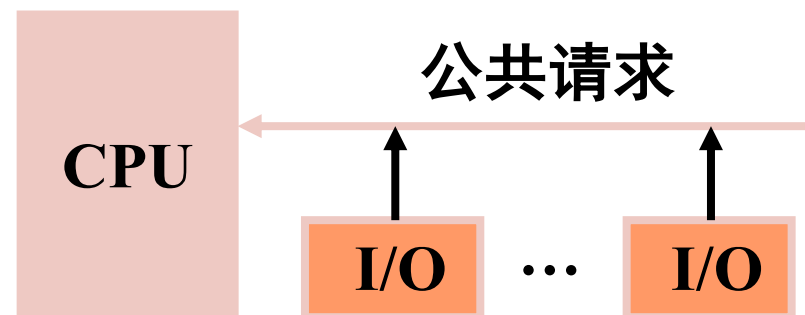
- 外设有中断请求需要： 例如准备就绪、任务完成等；
- 该中断请求未被屏蔽： 例如屏蔽标志为0



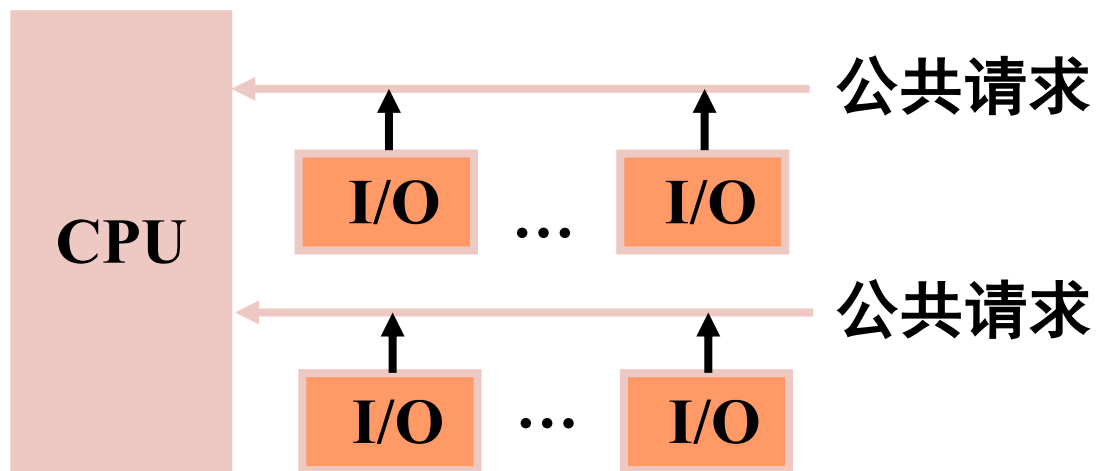
(2) 如何传送中断请求?



1) 使用单独请求线



2) 使用公共请求线



3) 混合方式传送

2. 中断请求优先级的判断

(1) 中断源优先级顺序的安排

(2) CPU当前程序与外设请求的判优

- 先查询CPU的允许中断标志 $\begin{cases} =1, & \text{开中断} \\ =0, & \text{关中断} \end{cases}$

- 再分析当前程序的优先级（PSW字段）

现行程序的优先级	$<$ 外设请求优先级,	响应
	$\geq$ 外设请求优先级,	不响应

划分优先级的一般规律

- 硬件故障中断属于最高级，其次是程序错误中断
- 非屏蔽中断优于可屏蔽中断
- DMA请求优先于I/O设备传送的中断请求
- 高速设备优于低速设备
- 输入设备的中断优于输出设备
- 实时设备优先于普通设备

[返回](#)

(3) 多个中断请求之间的判优

(1) 软件判优

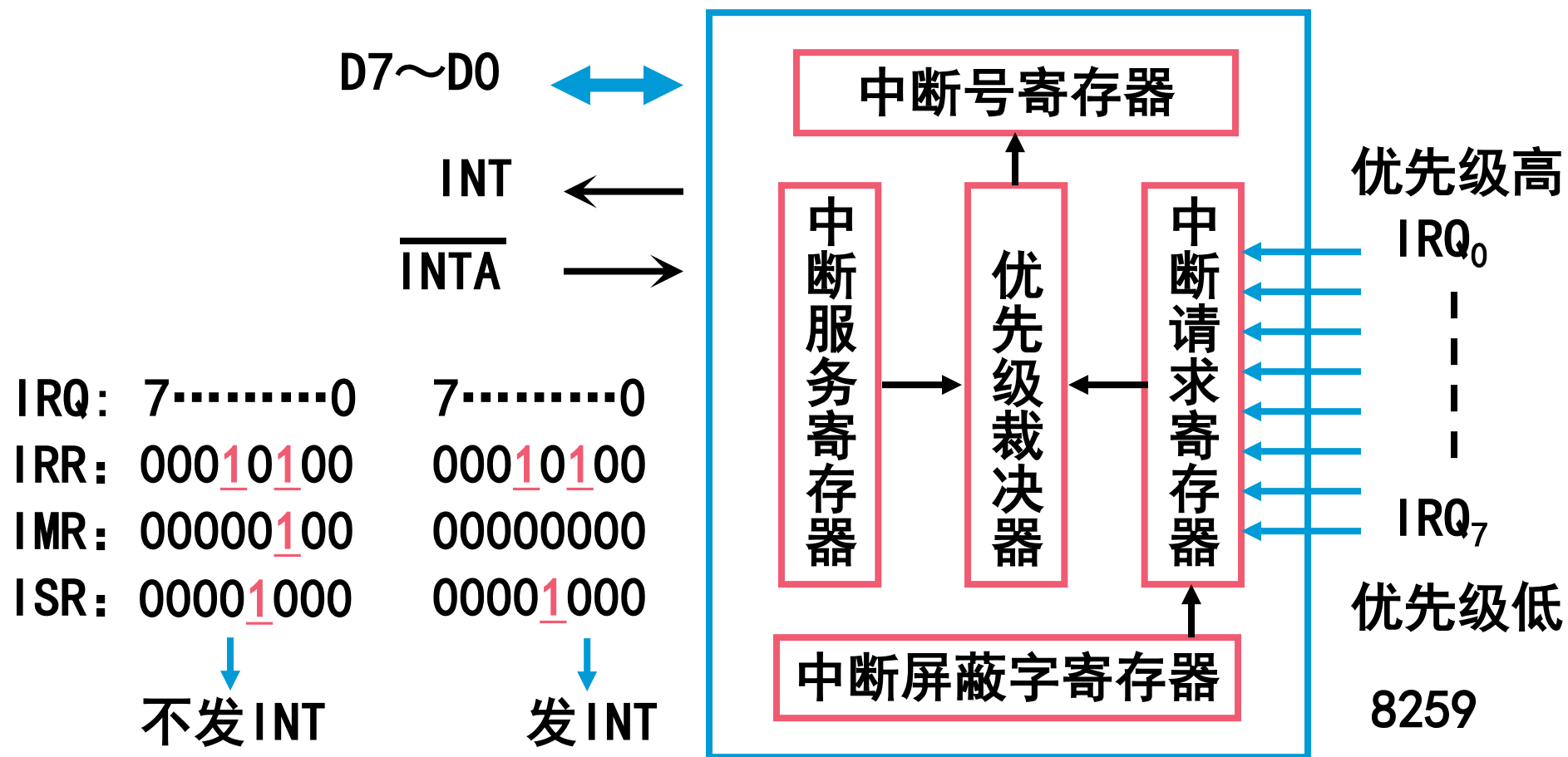
由程序查询顺序确定优先级。

可灵活修改优先级。

(2) 硬件判优

[例] 通过中断控制器判优

中断控制器(如8259)集中解决请求信号的接收、屏蔽、判优、编码等问题。



3. CPU对中断请求的响应

(1) 响应条件

- ✓有未被屏蔽中断请求到达；
- ✓CPU处于开中断模式；
- ✓中断源优先级比当前程序的优先级更高；
- ✓CPU刚执行完一条指令(非停机)；

(2) 如何形成中断服务程序的入口地址？

取决于中断方式是

{ 向量中断
非向量中断

非向量中断(程序识别)

- 将所有中断源的中断服务程序入口地址组织在**公共查询程序**中
- CPU响应时执行此查询程序，确定中断源对应的服务程序入口地址

向量中断(硬件查询法)

- 将所有中断源的中断服务程序入口地址(**中断向量**)组织在**中断向量表**中
- CPU响应时由硬件产生中断号**向量地址**，据此查**中断向量表**确定服务程序入口地址

- **中断向量：** 中断服务程序的入口地址+状态字PSW；
- **中断向量表：** 用来存放中断向量的表 (一段存储区)
- **向量地址：** 用来访问向量表的地址 (也叫**中断指针**)

4、中断的响应过程（向量中断方式）

执行中断隐指令 → 中断服务程序

※中断隐指令

CPU发INTA信号，进入IT

关中断、保存断点和PSW

取中断号，转换为向量地址，访问中断向量表

读取中断向量后，准备执行中断服务程序

硬件
自动完成

※中断服务程序的处理

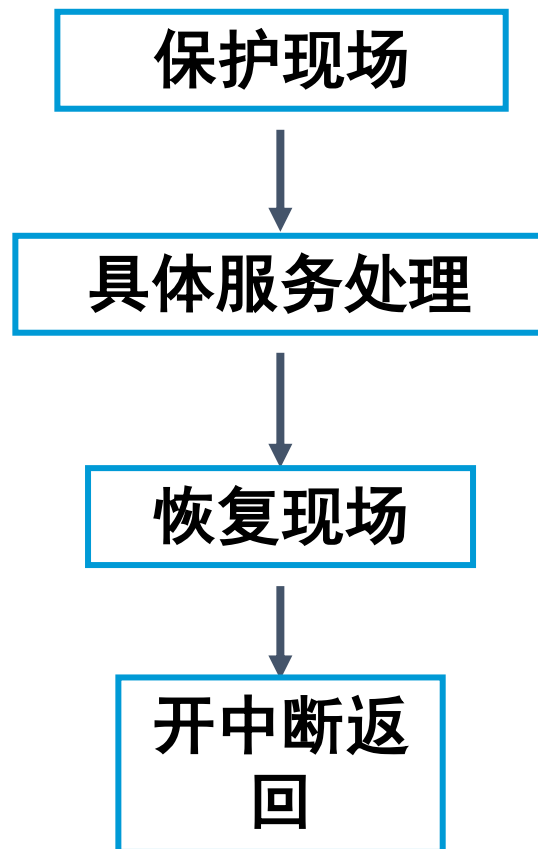
主要任务：CPU执行中断服务程序。

(1) 单级中断：CPU响应后只处理一个中断源的请求，处理完毕后才能响应新的中断请求。

(2) 多重中断：在某次中断服务过程中，允许响应处理更高优先级的中断请求。

	单级中断	多重中断
中断隐指令	关中断、保存断点及PSW等，取服务程序入口地址及新的PSW	关中断、保存断点及PSW等，取服务程序入口地址及新的PSW
中断服务程序	如后图所示	如后图所示

单级中断

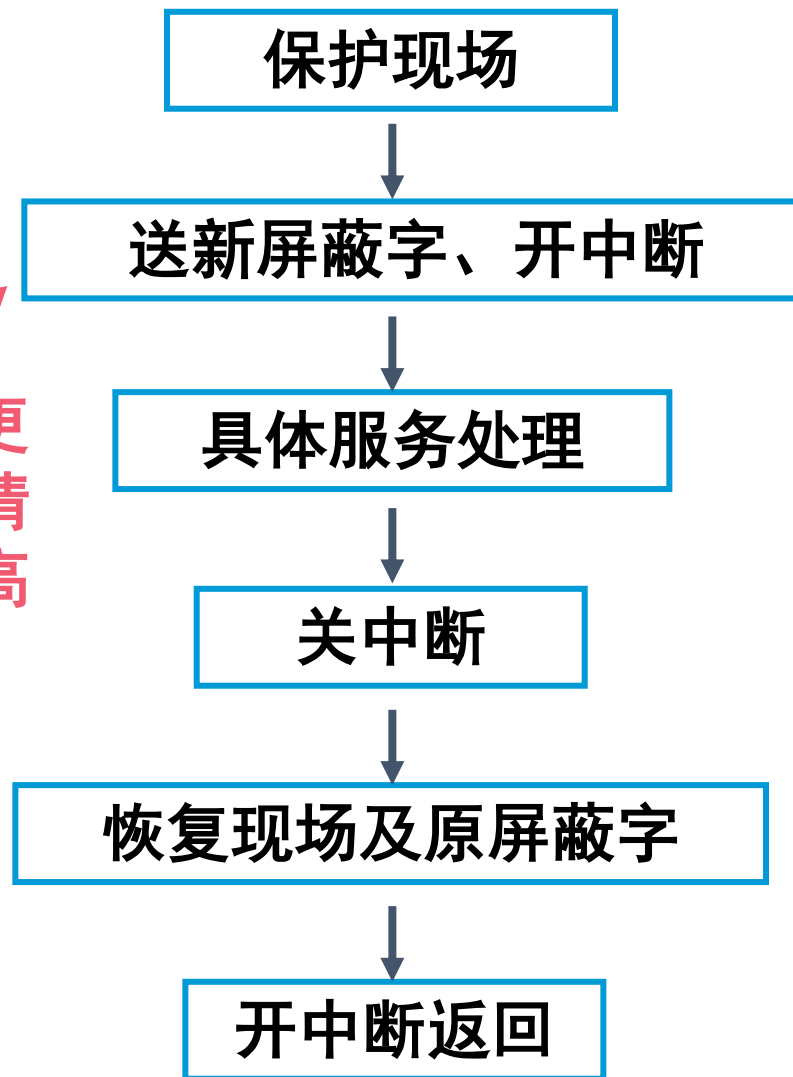


外中断:
数据传送

禁止同级或更低级的中断请求, 允许更高级的请求

【注意】 在响应过程、保护现场、恢复现场等关键阶段, 应关中断响应以防止被打扰。

多重中断



为什么要引入DMA方式

■ 程序直接控制方式

- 受“踏步”现象的限制，效率低下，不适合高速设备和主机间的数据传送

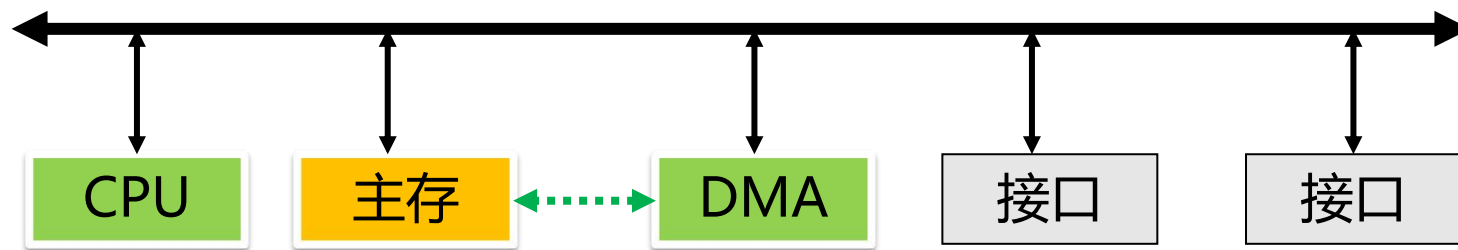
■ 中断控制方式

- 对I/O请求响应慢。每传送一个数据都要等待外设的中断请求，并增加许多中断响应和中断处理前、后的附加开销（保护断点、现场等），不能及时响应I/O请求。
- 数据传送速度慢。数据传送由软件完成（由CPU执行相应的中断服务程序来完成），速度慢。

DMA基本概念

■ DMA方式

- 全称直接存储器存取 (Direct Memory Access)
- 外设与主存间建立一个由硬件管理的数据通路 (虚拟通路, 还是通过系统总线)
- 直接依靠硬件系统来控制主存与外设之间的数据传送, 传送期间无需CPU干预
- 减少CPU开销, 提升效率
- 传送结束后通常用中断方式通知CPU



DMA方式适用场合

- 高速设备（如：磁盘、光盘等）
- 成批数据交换，且数据间间隔时间短，一旦启动，数据连续读写
- 动态存储器（DRAM）的自动刷新

DMA方式特点

■ 采用“请求-响应”方式

- 每当高速设备准备好数据，就进行一次DMA请求，DMA控制器接受到DMA请求后，申请总线使用权
- DMA控制器的总线使用优先级比CPU高

■ 与中断控制方式结合使用

- DMA传送前，寻道、旋转等操作结束时，通过中断告知CPU
- 在DMA控制器控制总线进行数据传送时，CPU执行其他程序
- DMA传送结束时，要通过DMA结束中断告知CPU

内存争用

■ 访存冲突

- DMA控制器直接访问内存
- CPU执行程序（需要访内）

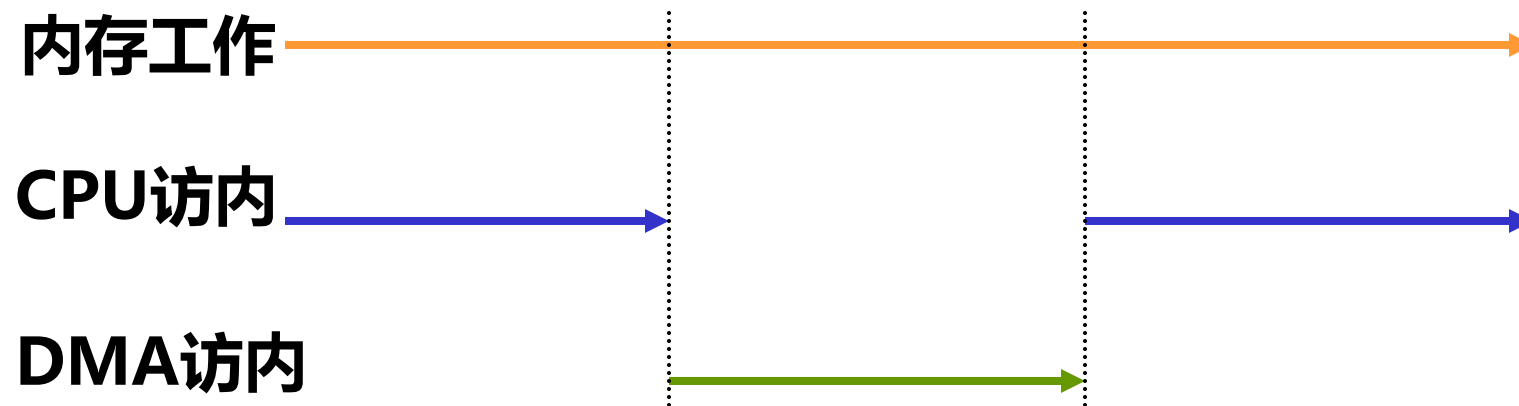
■ 如何处理这种冲突？

- 停止CPU使用主存(成组传送)
- DMA与CPU交替使用主存
- 周期挪用法(单字传送)

停止CPU使用主存

- DMA传送数据时，CPU停止使用主存
- 一批数据传送结束后，DMA再交还主存使用权
- DMA传送过程中，CPU处于等待状态

停止CPU使用主存



1. DMA批量数据传输周期过长，CPU长期无法访内
2. 外设传送两个数据的时间间隔大于存储周期，内存未充分利用

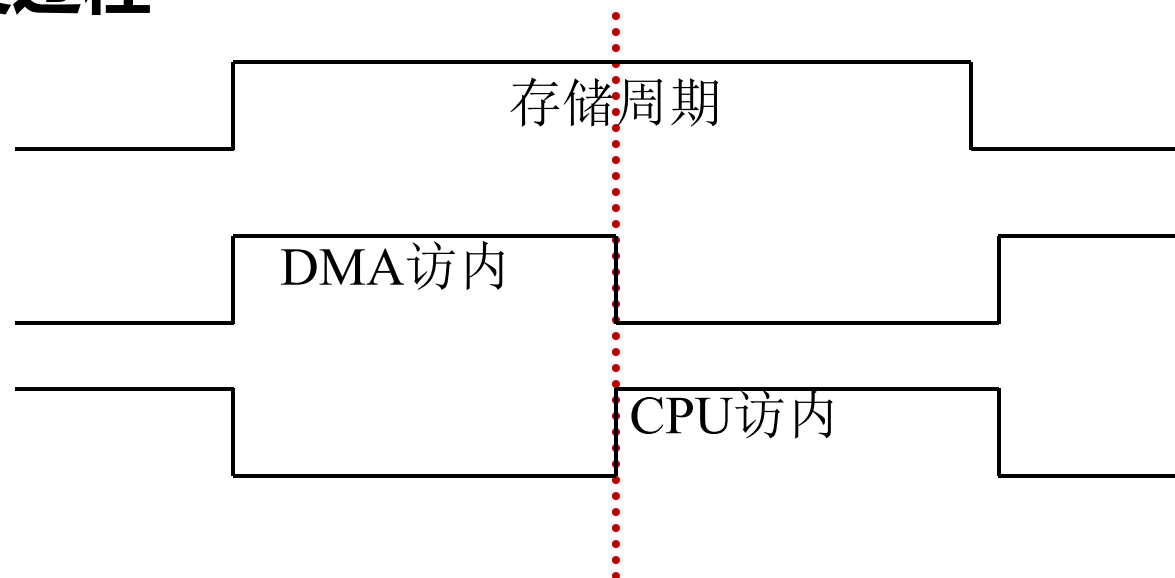


DMA与CPU交替使用主存

■ 每个存储周期分成两段

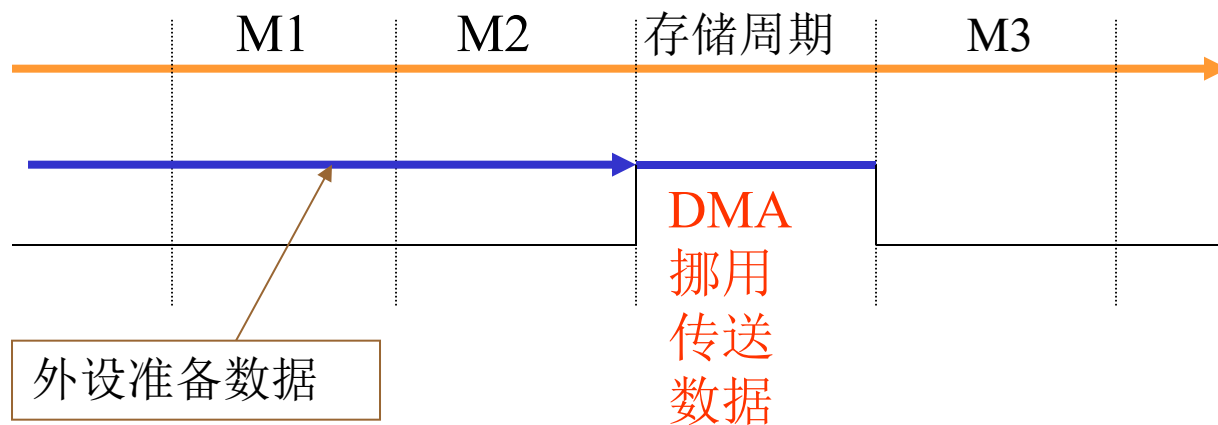
- 一段用于DMA访问主存
- 一段用于CPU访问主存

■ 无主存使用权移交过程



周期挪用法

- DMA要求访问主存时，CPU暂停一个或多个存储周期，由DMA控制总线来访问主存。一个数据传送结束后，CPU继续运行。
- CPU现场没有变动，仅延缓了指令的执行
 - 周期挪用，或称周期窃取。
- 如发生访存冲突，则DMA优先访问。



DMA控制器功能

■ DMA数据传送过程由DMA接口的控制逻辑完成，DMA接口也称DMA控制器

- **请求**：能接收外设发来的“DMA请求”信号，并能向CPU发“总线请求”信号。
- **响应**：当CPU发出“总线响应”信号响应请求后，能接管对总线的控制。
- **修改主存地址**：能在地址线上给出主存地址，并自动修改主存地址。
- **识别传送方向**：能识别传送方向以在控制线上给出正确的读写控制信息。
- **确定传送数据个数**
- **能发出DMA结束信号**：引起一次DMA中断，进行数据校验等一些后处理。

DMA控制器的操作步骤

■ DMA控制器的预置(初始化)----软件实现

□ 准备内存区

- * 输入：在内存设置好缓冲区

- * 输出：先在内存准备好数据

□ 设置传送参数

- 执行I/O指令，测试外设状态，对DMA控制器设置各种参数：

- * 内存首址=〉地址寄存器

- * 字计数值=〉字计数器

- * 传送方向=〉控制寄存器

- * 设备地址=〉设备地址寄存器

- 启动外设，然后执行其他程序

DMA控制器的操作步骤

■ DMA数据传送----硬件实现

- **DMA请求**: 选通→DMA请求→总线请求
- **DMA响应**: 总线响应(CPU让出总线)→DMA响应
- **DMA传送**: DMA控制总线进行数据传送

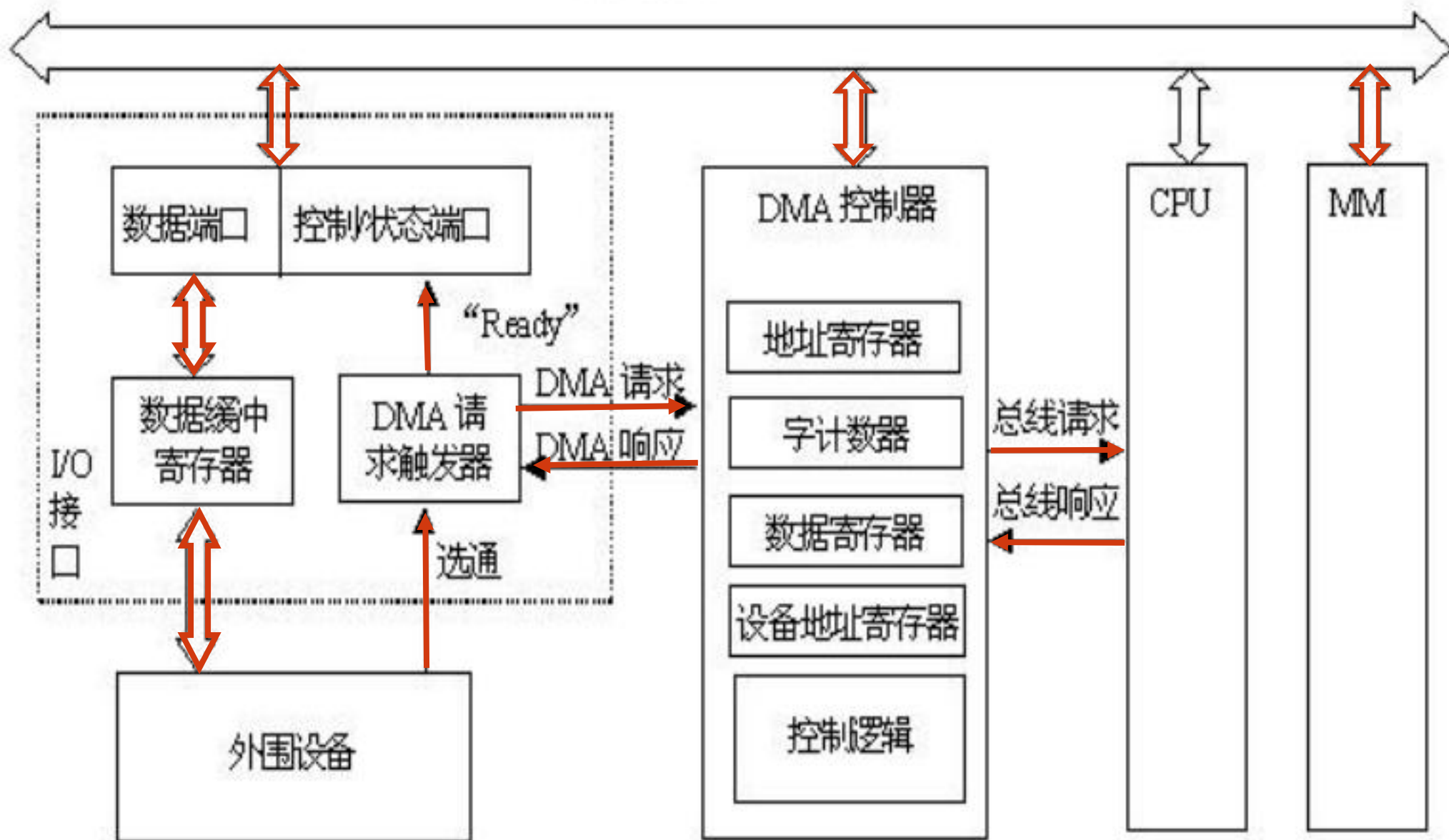
■ DMA结束处理----软件实现

- 根据计数值为0, 发出DMA结束信号去接口控制产生DMA中断请求信号, 转入中断服务程序, 做一些数据校验等后处理工作

DMA传输过程

- 当外设准备好数据或准备好接收数据时，发**选通**信号，使数据送数据缓冲寄存器，同时DMA请求触发器**置1**
- DMA请求触发器向控制/状态端口发**Ready**信号，同时向DMA控制器发**DMA请求**信号
- DMA控制器向CPU发**总线请求**信号
- CPU完成现行机器周期后，响应DMA请求，发出**总线响应**信号。DMA控制器接受到该信号后，发出**DMA响应**信号，使DMA请求触发器复位。此时，CPU浮动它的总线，让出总线控制权，由DMA控制器控制总线
- DMA控制器给出内存地址，并在其读/写线上发出“读”或“写”命令，随后在数据总线上给出数据
- 根据读写命令，将数据总线上的数据写入存储器中，或写入数据端口，并进行主存地址增量，计数值减1
- 若采用**CPU停止法**，则循环上一步，直到计数值为“0”
- 若采用**周期挪用法**，则释放总线

系统总线



DMA与程序中断的区别

- 中断通过程序(CPU执行中断服务程序)传送数据，DMA靠硬件(DMA控制器)来实现。
- 中断时机为两指令之间，DMA响应时机为两存储周期之间。
- 中断不仅具有数据传送能力，还能处理异常事件。DMA只能进行数据传送。
- DMA仅挪用了—个存储周期，不改变CPU现场。
- DMA请求的优先权比中断请求高。CPU优先响应DMA请求，是为了避免DMA所连接的高速外设丢失数据。
- DMA利用了中断技术

THANKS